How V Foods Works

1. The whole system, top to bottom

Status: Built.

The system connects customer phones to campus food outlets and payment services.

Flow chain: Phone -> Frontend -> Backend -> Database -> Paytm

1. The customer opens the web application on a mobile browser. The web app is a PWA. A PWA is a progressive web app. It is a website that installs like an application.
2. The browser loads outlet profiles and food catalogs directly from Supabase.
3. The browser cannot change food prices. The database stops unauthorized price changes using RLS. RLS means row-level security. These database rules filter rows per user. Database stored procedures enforce price math.
4. The customer places an order using their prepaid campus wallet balance.
5. The frontend calls the backend API. An API is an application programming interface. It lets programs talk to each other.
6. The backend connects to PostgreSQL using the secret Supabase service role key.
7. The database processes this action as a transaction. A transaction is a group of database changes. They succeed or fail together as one unit.
8. Realtime pushes the new order to kitchen screens. Realtime means instant data updates sent over an open connection.
9. For direct online payments, the backend signs payment parameters using HMAC-SHA256.
10. The browser forwards the customer to Paytm.
11. The customer approves the payment on Paytm.
12. Paytm calls a backend webhook. A webhook is an automated HTTP message sent when an event happens.
13. The backend checks the cryptographic checksum of the webhook.
14. The backend confirms the order in the database.
15. The database runs the three-way split calculation.
16. The backend sends order status updates to customer phones using MSG91 WhatsApp messages.
17. The frontend never talks directly to Paytm. Paytm talks only to the backend webhook route.

> **Checked:** frontend/src/App.jsx, backend/main.py, supabase/migrations/001_initial_schema.sql, supabase/migrations/011_paytm_three_way_split.sql, tables: outlets, menu_items, orders, payments, wallets, profiles.

2. The four roles

Status: Built.

The platform provides four separate user roles. Permissions are enforced by PostgreSQL RLS policies and backend dependencies.

Role	Status	What it can do	What it cannot do	How account is made
User	Built	Browse canteens; search dishes; add items to cart; order via wallet or gateway; top up wallet; track order status; rate collected food; edit own profile.	Cannot change menu prices; cannot view other customer orders; cannot view other wallets; cannot access staff screens.	Created by self sign-up with email OTP, Google OAuth, or automatic creation via database trigger `_handle_new_user`.
Shop Staff	Built	View live incoming orders for assigned outlet; advance orders from placed to preparing and ready; verify pickup tokens; adjust dish stock quantities; create walk-in counter POS orders.	Cannot change dish prices; cannot archive dishes; cannot invite staff; cannot view or edit other outlets.	Created by accepting an invite code using database function `accept_invite` or backend route `/invites/accept`.
Shop Admin	Built	Perform all staff actions; edit food items; change item prices; archive dishes; toggle outlet open status; invite staff; view outlet sales reports; view franchise settlements.	Cannot access or edit other outlets; cannot change platform split percentages; cannot alter global platform settings.	Created by accepting an invite code generated for the `shop_admin` role, or assigned in table `profiles`.
Super Admin	Built	View all campus outlets; view all platform orders; manage all user accounts; create coupons; inspect financial splits; view security audit logs; update platform settings.	Cannot bypass business logic rules hardcoded in database functions; no RLS rule blocks this role.	Assigned by direct database record update on column `profiles.role` to `super_admin`.

> **Checked:** supabase/migrations/008_four_tier_roles_invites_and_stock.sql, supabase/migrations/009_dashboard_backend_unification.sql, frontend/src/App.jsx, backend/main.py, tables: profiles, orders, menu_items, invites.

3. Frontend

Status: Built.

The frontend is a single-page application built with React, Vite, TypeScript, and Tailwind CSS. It serves all four user roles from one codebase.

What screens exist for each role

- User role screens: User screens show campus canteen lists and dish menus. They provide item search and a shopping cart. They show active orders with tokens and wallet ledgers. Customers edit personal profile settings here.
- Shop Staff screens: Staff screens display live incoming orders and a kitchen board. Staff can toggle item stock and scan pickup tokens. The screens support counter POS billing and shift summary reports.
- Shop Admin screens: Shop admin screens show orders, menu editors, dish prices, and inventory. Admins manage staff rosters, pickup windows, and store open toggles. Screens also display revenue charts, settlement shares, and business exports.
- Super Admin screens: Super admin screens display campus hubs, outlet hierarchy, and user accounts. They show platform order lists and the full payment ledger. Admins manage coupons, franchise splits, security audit logs, and system settings.

Routing

Application routing is handled by React Router in `frontend/src/App.jsx`. Path boundaries: - Public customer routes: `/`, `/browse`, `/canteen/:id`, `/search`, `/orders`, `/cart`, `/wallet`, `/profile`. - Shop staff routes: `/staff`, `/staff/orders`, `/staff/kds`, `/staff/menu`, `/staff/scanner`, `/staff/pos`, `/staff/summary`. - Shop admin routes: `/shop-admin`, `/shop-admin/orders`, `/shop-admin/menu`, `/shop-admin/pos`, `/shop-admin/slots`, `/shop-admin/staff`, `/shop-admin/analytics`, `/shop-admin/reports`. - Super admin routes: `/admin`, `/admin/users`, `/admin/orders`, `/admin/analytics`, `/admin/coupons`, `/admin/settlements`, `/admin/audit`, `/admin/settings`.

Role-based access guard `RoleRoute` verifies authenticated user roles before rendering dashboards. Unauthorized requests redirect to home.

Mobile app status

The web frontend is configured as a Progressive Web App (PWA). A web app manifest exists at `frontend/public/manifest.json`. A custom service worker exists at `frontend/public/sw.js`. The package `@capacitor/haptics` is installed for touch vibration feedback. Native Android and iOS project folders do not exist.

Why chosen and why not

- Frontend, React with Vite, TypeScript and Tailwind. Why React with Vite: One code base serves all four dashboards. The application runs in browsers and installs as a PWA. Supabase client libraries and UI tools are ready to use. TypeScript catches mistakes in order and wallet screens before release. Why not Flutter: Flutter requires Dart and a second toolchain. Web build files are larger, making staff screens load slower. The frontend would restart from zero. Why not plain HTML and JavaScript: Plain JavaScript lacks type checking for money calculations. It prevents code reuse across the four separate dashboards. Why not Next.js: Every screen needs login, so search-engine rendering is not needed. Why not React Native: Every screen would be rebuilt. Phone install is by PWA from a link. Capacitor configuration exists without native folders.

> **Checked:** frontend/package.json, frontend/vite.config.ts, frontend/capacitor.config.json, frontend/public/manifest.json, frontend/public/sw.js, frontend/src/main.jsx, frontend/src/App.jsx.

4. Backend

Status: Built.

Table of every route

Route path	Who calls it	What it does	Status
`GET /`	Web browser or monitor	Returns API service name, health link, and documentation link.	Built
`GET /health`	Uptime checker	Returns process liveness and current UTC timestamp.	Built
`GET /health/live`	Uptime checker	Verifies process is alive and responsive.	Built
`GET /health/ready`	Deployment health monitor	Queries database to verify connection pool readiness.	Built
`GET /api/catalog/summary`	Customer frontend	Returns in-memory cached list of open outlets with 30-second TTL.	Built

Route path	Who calls it	What it does	Status
`POST /wallet/topup`	Authenticated user	Creates legacy PhonePe wallet top-up payment order.	Built
`POST /wallet/topup/paytm`	Authenticated user	Inserts pending payment record and generates signed Paytm top-up payload.	Built
`POST /order/checkout`	Authenticated user	Deducts wallet balance atomically or records gateway order payment.	Built
`POST /order/checkout/gateway-session`	Authenticated user	Creates pending order and payment intent for PhonePe payment.	Built
`POST /order/checkout/paytm-session`	Authenticated user	Creates pending order, payment row, and returns signed Paytm payload.	Built
`POST /webhooks/phonepe`	PhonePe payment server	Verifies signature and credits wallet or finalizes gateway order.	Built
`POST /webhooks/paytm`	Paytm payment gateway	Verifies checksum, updates payment status, and executes split.	Built
`POST /staff/order/{order_id}/advance`	Staff, Shop Admin, Super Admin	Advances order status from placed to preparing or ready.	Built
`POST /staff/order/{order_id}/cancel`	Staff, Shop Admin, Super Admin	Cancels order and triggers refund if paid from wallet.	Built
`POST /staff/scan`	Staff, Shop Admin, Super Admin	Validates HMAC signature or daily token and marks order collected.	Built
`POST /staff/stock/adjust`	Staff, Shop Admin, Super Admin	Updates dish inventory quantity and writes stock adjustment log.	Built
`POST /admin/credit-wallet`	Super Admin	Credits funds directly to customer wallet with audit note.	Built
`POST /admin/invites/create`	Shop Admin or Super Admin	Generates random 8-character invite code for new staff.	Built
`POST /invites/accept`	Authenticated user	Validates code, assigns user role, and assigns outlet reference.	Built
`POST /media/upload-menu-image`	Staff, Shop Admin, Super Admin	Uploads food photo to Supabase Storage with local file fallback.	Built
`POST /api/auth/send-otp`	Unauthenticated user	Generates 6-digit email verification code with 10-minute expiry.	Built
`POST /api/auth/verify-otp`	Unauthenticated user	Checks submitted code and returns verification token.	Built

What it never does

- The backend never serves server-rendered HTML pages.
- The backend never stores credit card numbers, CVVs, or bank PINs.
- The backend never allows unverified webhook payloads to modify data.

Where secrets live

Secrets live in Render environment settings and local `.env` configuration files. Variable names: - `SUPABASE_URL` - `SUPABASE_SERVICE_ROLE_KEY` - `SUPABASE_JWT_SECRET` - `PAYTM_MID` - `PAYTM_MERCHANT_KEY` - `PAYTM_WEBSITE` - `PAYTM_CHANNEL_ID` - `PAYTM_INDUSTRY_TYPE_ID` - `PAYTM_ENV` - `PHONEPE_MERCHANT_ID` - `PHONEPE_SALT_KEY` - `PHONEPE_SALT_INDEX` - `PHONEPE_ENV` - `PHONEPE_BASE_URL` - `WEBHOOK_BASE_URL` - `FRONTEND_URL` - `QR_SECRET` - `MSG91_AUTH_KEY` - `MSG91_WHATSAPP_INTEGRATED_NUMBER` - `MSG91_OTP_TEMPLATE_ID` - `MSG91_SENDER` - `SMTP_HOST` - `SMTP_PORT` - `SMTP_USER` - `SMTP_PASSWORD` - `SMTP_FROM`

Why chosen and why not

- Backend, FastAPI (Python) on Render. Why FastAPI on Python: FastAPI holds secret keys away from customer phones. It signs Paytm requests and checks signed webhook messages. It sends customer notifications through external services. It validates request data and generates API documentation automatically. The service deploys directly on Render. Why not Node.js with Express: Node.js with Express would also work. FastAPI provides request validation and API documentation out of the box. Why not Django: Its database layer and admin panel repeat what Supabase already gives. Why not Supabase Edge Functions only: The owner judged signed Paytm messages easier to control in standard servers. Handling webhook retries is more manageable in a dedicated Python backend. Edge Function verification: 0 Edge Functions exist in the Supabase project.

> **Checked:** backend/main.py, backend/routers/auth.py, backend/routers/media.py, backend/requirements.txt, backend/render.yaml, backend/.env.example.

5. Database

Status: Built.

Every table with one sentence

1. `audit_logs` : Stores platform security events, administrative role updates, and user action records.
2. `college_settlement_accounts` : Stores campus college Paytm account details and facility split configurations.
3. `coupon_redemptions` : Stores records linking redeemed coupons to orders and user accounts.
4. `coupons` : Stores promotional discount codes, percentage or flat values, and validity schedules.
5. `invites` : Stores one-time registration codes for onboarding staff and shop admins.
6. `item_ratings` : Stores customer reviews and star ratings for collected orders.
7. `loyalty_progress` : Stores order counts and reward item redemption counters per user.
8. `menu_items` : Stores food dishes, prices, availability, time windows, and stock quantities.
9. `menu_items_audit` : Stores field-level historical records of changes made to menu items.
10. `notifications` : Stores notification messages dispatched to students and staff.
11. `order_items` : Stores itemized dishes, quantities, and price snapshots for each order.
12. `orders` : Stores purchase records, statuses, payment methods, totals, tokens, and daily numbers.
13. `outlet_daily_counters` : Stores daily sequential counters per outlet for order number generation.
14. `outlet_paytm_accounts` : Stores shop Paytm merchant account details and settlement split percentages.
15. `outlets` : Stores campus food stalls, locations, open statuses, and event stall flags.
16. `payment_splits` : Stores calculated split shares for the shop, platform, and college.
17. `payments` : Stores transaction intents, statuses, payment methods, amounts, and gateway references.

18. `payments_legacy`: Stores earlier PhonePe payment transaction records.
19. `pickup_slots`: Stores scheduled pickup times and order capacity limits per outlet.
20. `platform_settlement_account`: Stores platform Paytm merchant details and revenue split percentages.
21. `profiles`: Stores user names, phone numbers, roles, and assigned outlet references.
22. `push_subscriptions`: Stores Web Push browser endpoints and client encryption keys.
23. `refunds`: Stores gateway and wallet refund transaction details.
24. `settings`: Stores global platform flags, including event mode.
25. `stock_adjustments`: Stores audit entries for every inventory increase or decrease.
26. `wallet_txns`: Stores every ledger entry for top-ups, orders, refunds, and credits.
27. `wallets`: Stores user prepaid account balances.

How a shop is tied to its menu, orders and staff

- Table `menu_items` contains column `outlet_id` referencing table `outlets.id`.
- Table `orders` contains column `outlet_id` referencing table `outlets.id`.
- Table `profiles` contains column `outlet_id` referencing table `outlets.id`.

Two real access-rule examples from the policies

1. Policy `orders_student_read` on table `orders`. Expression: `(auth.uid() = user_id)`. The database refuses SELECT requests when the logged-in user ID does not match `user_id`.
2. Policy `menu_items_shop_admin` on table `menu_items`. Expression: `((get_my_role() = 'shop_admin') AND (outlet_id = get_my_outlet()))`. The database refuses write requests when the dish does not belong to the user's outlet.

Every database function and what it does

1. `_generate_token(p_outlet_id)`: Generates daily sequential order token for outlet.
2. `_handle_new_user()`: Trigger function creating profile and zero-balance wallet on signup.
3. `_set_updated_at()`: Trigger function setting column `updated_at` to current timestamp.
4. `accept_invite(p_code)`: Validates invite code and assigns profile role and outlet.
5. `adjust_stock(p_item_id, p_new_qty, p_reason)`: Updates item inventory quantity and logs stock adjustment entry.
6. `allocate_order_number(p_outlet_id)`: Atomically increments daily order counter and returns formatted number.
7. `calculate_order_split(p_order_id, p_payment_id)`: Divides order payment between shop, platform, and college accounts.
8. `check_item_rating_order_status()`: Trigger refusing dish ratings unless order status is collected.
9. `check_mobile_availability(p_mobile, p_user_id)`: Verifies if a mobile number is unassigned in profiles.
10. `complete_google_profile(p_mobile)`: Saves verified mobile number on Google OAuth user profile.
11. `create_counter_order(p_outlet_id, p_items, p_payment_method)`: Creates counter order, allocates token, and decrements stock.
12. `create_google_test_user(p_email, p_name)`: Creates test user account in auth and public profiles.
13. `create_invite(p_role, p_outlet, p_email, p_phone)`. Generates staff invite code.

14. `create_pending_gateway_order(p_user_id, p_outlet_id, p_items)`: Reserves stock and inserts pending order with 10-minute expiry.
15. `create_pos_order(p_outlet_id, p_items, p_payment_mode, p_notes)`. Creates walk-in POS order. Allocates token and decrements stock.
16. `credit_wallet(p_user_id, p_amount, p_kind, p_ref, p_note)`. Adds balance to user wallet. Inserts ledger transaction.
17. `expire_pending_orders()`: Cancels expired pending orders and releases reserved item stock.
18. `expire_stale_awaiting_orders(p_older_than_minutes)`: Cancels stale unpaid orders and associated pending payment records.
19. `finalize_gateway_order(p_order_id, p_phonepe_txn_id)`: Confirms paid gateway order, decrements stock, and assigns token.
20. `generate_daily_pickup_slots(p_outlet_id, p_interval, p_max, p_start, p_end)`. Inserts scheduled pickup slots for outlet.
21. `get_my_outlet()`: Returns outlet ID assigned to currently authenticated user.
22. `get_my_role()`: Returns normalized role string for currently authenticated user.
23. `get_outlet_analytics(p_outlet_id)`: Returns aggregated revenue, order count, and top-selling dishes.
24. `get_super_admin_overview()`: Returns global platform GMV, profit, outlet tree, and audit logs.
25. `lookup_email_by_mobile(p_mobile)`: Finds email address associated with a registered mobile number.
26. `normalize_phone(p_raw)`: Strips non-digit characters and extracts standard 10-digit Indian phone number.
27. `place_order(p_user_id, p_outlet_id, p_items, p_payment_method)`. Dispatches order placement to wallet checkout function.
28. `place_order_wallet(p_user_id, p_outlet_id, p_items)`: Deducts wallet balance, decrements stock, and places order atomically.
29. `place_order_wallet(p_user_id, p_outlet_id, p_items, p_slot, p_coupon)`. Validates fees and coupons. Debits wallet and allocates order token.
30. `record_refund(p_payment_id, p_refund_ref, p_refund_type, p_amount, p_status)`. Upserts refund transaction row. Marks payment refunded.
31. `register_manual_user(p_name, p_email, p_mobile, p_password)`. Creates authenticated user credentials. Initializes user wallet account.
32. `revoke_invite(p_invite_id)`: Marks pending staff invite status as revoked.
33. `rls_auto_enable()`: Event trigger function automatically enabling row-level security on new tables.
34. `set_staff_active_status(p_staff_id, p_is_active)`: Updates active duty status of staff member profile.
35. `topup_my_wallet(p_amount, p_payment_ref)`: Allows authenticated user to add balance to own wallet.
36. `trg_assign_order_number()`: Trigger keeping order token and order number synchronized on insert.
37. `trg_log_menu_item_changes()`: Trigger writing field-level price, name, and availability changes to audit table.
38. `trg_menu_items_staff_restrictions()`: Trigger stopping staff from changing dish prices or archiving items.
39. `trg_observe_order_notifications()`: Trigger creating notifications when orders are placed, ready, or cancelled.
40. `trg_orders_status_update_order_number()`: Trigger assigning daily order number when order payment completes.
41. `trg_prevent_hard_delete_menu_items()`: Trigger blocking table deletion of dishes that have existing order records.
42. `update_order_status(p_order_id, p_status)`: Updates status column on orders table.

43. `verify_and_collect_order(p_order_id, p_token)`: Verifies token match and transitions order status to collected.
44. `verify_and_record_payment(p_paytm_txn_id, p_status, p_order_id, p_ref)`. Processes Paytm webhook. Confirms order and records splits.
45. `verify_pickup_code(p_outlet_id, p_order_id, p_token)`: Validates pickup token and marks order status collected.
46. `void_pos_order(p_order_id, p_reason)`: Cancels walk-in POS order and restores menu item stock.

The order journey from "Place order" to "Collected"

1. Customer taps place order. Database function `place_order_wallet` executes. Table `wallets` balance is debited. Table `orders` receives new row with status placed. Table `order_items` receives item snapshot rows. Table `menu_items` stock quantity is decremented. Table `stock_adjustments` receives audit log row. Table `wallet_txns` receives debit transaction row. Table `payments` receives payment record. Table `payment_splits` receives split breakdown rows.
2. Kitchen staff receives live order. Staff changes order status to preparing. Table `orders` column `status` is updated.
3. Kitchen staff finishes food preparation. Staff changes order status to ready. Table `orders` column `status` is updated. Trigger `trg_observe_order_notifications` inserts a row into table `notifications`.
4. Customer arrives at counter. Staff verifies token or scans QR code. Database function `verify_and_collect_order` executes. Table `orders` column `status` is updated to collected.
5. Customer submits rating for food item. Table `item_ratings` receives new row. Trigger `check_item_rating_order_status` verifies order status is collected before saving.

How the order number is made, how stock changes, and where the money math runs

- Order number: Database function `allocate_order_number(p_outlet_id)` locks table `outlet_daily_counters` for today's date. It increments `last_number` and formats it as a 3-digit string.
- Stock changes: Handled inside database functions `place_order_wallet`, `create_pos_order`, and `verify_and_record_payment`. The functions decrement `menu_items.stock_qty` and insert rows into `stock_adjustments`.
- Money math: Calculations run inside database functions `place_order_wallet`, `create_pos_order`, and `calculate_order_split`. The frontend cannot set prices or fee totals.

Why chosen and why not

- Database, Supabase (PostgreSQL). Why Supabase with Postgres: Wallet and order changes succeed or fail together as a single transaction. Price and fee calculations run in database functions. Customer phones cannot alter prices. Row-level security stops unauthorized access to protected rows. Realtime pushes database updates to open browser screens. User authentication is built directly into the database engine. Why not Firebase or Firestore: Atomic wallet debits and order creation are harder in document stores. Firestore lacks advanced SQL stored functions. Why not MongoDB: Multi-document transactions are heavier than PostgreSQL row locks. Strict non-negative wallet constraints are better enforced in relational schemas. Why not self-hosted PostgreSQL with custom WebSockets: Self-hosting requires building custom authentication and connection pools. It requires maintaining custom WebSocket servers for live order updates.

> **Checked:** supabase/migrations/001_initial_schema.sql through 020_assign_orphan_outlets.sql, pg_proc, pg_policies, pg_tables, project wahftohnwfoepuszvzrx.

6. Realtime

Status: Built.

PostgreSQL Realtime publishes table changes to subscribed browser clients.

Published tables

The publication `supabase_realtime` publishes changes from: - `menu_items` - `wallets` - `orders` - `order_items` - `invites` - `pickup_slots` - `stock_adjustments` - `payments` - `refunds`

Channels and recipients

1. To Customer: Channel `vfoods-student-{userId}` receives order updates when `user_id = {userId}`. Channel `vfoods-global-menu-sync` receives dish availability and stock updates.
2. To Kitchen Staff: Channel `vfoods-outlet-{outletId}` receives new orders when `outlet_id = {outletId}`.
3. To Shop Admin: Channel `vfoods-outlet-{outletId}` receives orders and menu updates for that store.
4. To Super Admin: Channel `vfoods-superadmin-all` receives all order events across all campus outlets. Channel `super-admin-audit-stream` receives INSERT events from table `audit_logs`.

> **Checked:** frontend/src/App.jsx, frontend/src/pages/admin/SuperAdminDashboard.tsx, pg_publication, table `orders`.

7. Login and accounts

Status: Built.

Sign-up and login methods

1. Email sign-up with OTP: Customer enters name, email, mobile number, and password. Frontend calls `supabase.auth.signInWithOtp` to send a 6-digit verification code. Customer submits code. Frontend verifies code using `supabase.auth.verifyOtp`. Frontend calls database procedure `register_manual_user`. Procedure creates auth user in `auth.users`, creates profile, and creates wallet.
2. Google OAuth sign-up and login: Customer clicks Google sign-in. Frontend calls `supabase.auth.signInWithOAuth({ provider: 'google' })`. Database trigger `_handle_new_user` creates profile and zero-balance wallet. If mobile number is missing, customer must complete it via `complete_google_profile`.
3. Email and password login: Customer logs in via `supabase.auth.signInWithPassword`.
4. Mobile number and password login: Customer enters mobile number and password. Frontend calls stored function `lookup_email_by_mobile` to find email address. Frontend then signs in using `supabase.auth.signInWithPassword`.
5. Password reset: Frontend looks up email by mobile number. Frontend calls `supabase.auth.resetPasswordForEmail`.

How invites work

1. Shop Admin or Super Admin calls function `create_invite` or backend route `/admin/invites/create`.
2. Database generates an 8-character uppercase alphanumeric code in table `invites`.
3. New staff member signs up or logs in.
4. Staff member submits invite code to function `accept_invite` or route `/invites/accept`.

5. Database checks code status is pending and code has not expired.
6. Database updates `profiles.role` and `profiles.outlet_id`.
7. Database updates invite status to accepted.

> **Checked:** frontend/src/App.jsx, backend/routers/auth.py, backend/main.py, stored procedures: register_manual_user, create_google_test_user, complete_google_profile, lookup_email_by_mobile, create_invite, accept_invite, tables: profiles, invites.

8. Paytm, step by step

Status: Built.

a. Wallet top-up

1. Customer enters top-up amount and taps Add Money.
2. Frontend calls backend route `POST /wallet/topup/paytm`.
3. Backend checks minimum amount requirement.
4. Backend creates pending row in table `payments` with `payment_purpose = 'WALLET_TOPUP'`.
5. Backend builds parameter dictionary and generates HMAC-SHA256 signature using `PAYTM_MERCHANT_KEY`.
6. Backend returns gateway URL and signed parameter payload to browser.
7. Browser forwards customer to Paytm payment screen.
8. Customer completes UPI or card payment.
9. Paytm posts transaction payload to backend route `POST /webhooks/paytm`.
10. Backend verifies webhook checksum.
11. Backend calls database function `verify_and_record_payment`.
12. Database function calls `credit_wallet`.
13. Balance increases in table `wallets` and row inserts into table `wallet_txns`.
14. Realtime pushes updated wallet balance to customer screen.

b. Direct order payment

1. Customer selects instant gateway checkout and chooses Paytm.
2. Frontend calls backend route `POST /order/checkout/paytm-session`.
3. Backend checks IP rate limit and idempotency header.
4. Backend allocates sequential order token.
5. Backend inserts row into table `orders` with status `payment_pending`.
6. Backend inserts row into table `payments` with `payment_purpose = 'ORDER'`.
7. Backend signs parameters with HMAC-SHA256 and returns gateway URL.
8. Customer completes payment on Paytm payment page.
9. Paytm posts transaction result to backend route `POST /webhooks/paytm`.
10. Backend checks signature and calls database function `verify_and_record_payment`.
11. Database checks inventory availability for each ordered item.
12. Database decrements item stock in table `menu_items` and logs `stock_adjustments`.
13. Database updates order status to `confirmed`.
14. Database calls `calculate_order_split` to divide payment.

15. Realtime pushes new order to kitchen staff dashboard.

c. Order paid from the wallet

1. Customer taps place order with payment method set to wallet.
2. Frontend calls database function `place_order_wallet` directly or through backend route `/order/checkout`.
3. Database checks outlet open status and dish availability windows.
4. Database calculates item subtotals, tax, service charges, and convenience fees.
5. Database validates coupon code and calculates discount amount.
6. Database checks wallet balance covers payable total.
7. Database decrements wallet balance in table `wallets`.
8. Database allocates daily sequential token from table `outlet_daily_counters`.
9. Database inserts order record into table `orders` with status placed.
10. Database inserts item snapshots into table `order_items`.
11. Database decrements stock in table `menu_items` and logs table `stock_adjustments`.
12. Database logs transaction into table `wallet_txns`.
13. Database calls `calculate_order_split`.
14. Realtime pushes order to kitchen staff screen.
15. Paytm is not called.

d. The webhook

- Route: `POST /webhooks/paytm`.
- Check: Backend extracts checksum from request body or header. Backend sorts parameters and computes HMAC-SHA256 signature using `PAYTM_MERCHANT_KEY`. Backend validates signature using constant-time string comparison `hmac.compare_digest`. Invalid signatures return HTTP 400.
- Duplicate arrival: Handled idempotently. Database function `verify_and_record_payment` checks if payment status is already `SUCCESS` or `REFUNDED`. If already processed, it returns status `already_processed` immediately. It does not credit wallet or decrement stock twice.

e. Payment status check and refunds

- Payment status check: Not built. Backend has no background polling worker querying Paytm transaction status API.
- Refunds: Database refund tracking: Built. Table `refunds` and stored procedure `record_refund` exist. Wallet order cancellation refunds: Built. Backend route `/staff/order/{order_id}/cancel` automatically refunds wallet orders via `credit_wallet`. Gateway automated refunds: Not built. Backend code contains a TODO comment to connect Paytm refund API.

f. The three-way split

- Receivers: Shop, Platform (V Foods), College (Campus).
- When recorded: Recorded inside PostgreSQL function `calculate_order_split` when order payment is confirmed.
- Storage locations: Shares are stored in table `payment_splits`. Columns storing shares: `split_amount`, `split_percentage`, `recipient_type`, `outlet_paytm_account_id`, `platform_account_id`, `college_account_id`. Configured split shares are stored in:

- Table `outlet_paytm_accounts`, column `shop_split_percentage`.
- Table `platform_settlement_account`, column `platform_split_percentage`.
- Table `college_settlement_accounts`, column `college_split_percentage`.

g. What is stored and what is never stored

- Stored: `paytm_order_id`, `paytm_txn_id`, payment status, payment method, payment amounts, transaction timestamps, bank reference strings.
- Never stored: Card numbers, CVV security codes, and card expiration dates are never stored. UPI PINs and banking passwords are never handled.

h. What must exist outside the code

- Paytm merchant account: Must exist with Paytm. Status: Cannot confirm from code.
- Webhook address registered in Paytm dashboard: Must point to backend webhook route. Status: Cannot confirm from code.
- Environment variables: `PAYTM_MID`, `PAYTM_MERCHANT_KEY`, `PAYTM_WEBSITE`, `PAYTM_CHANNEL_ID`, `PAYTM_INDUSTRY_TYPE_ID`, `PAYTM_ENV`, `WEBHOOK_BASE_URL`. Confirmed present in code.

i. Test mode or live mode

Backend checks variable `PAYTM_ENV`. When set to `STAGE`, backend connects to staging URL `https://securegw-stage.paytm.in`. Otherwise it connects to production URL `https://securegw.paytm.in`. Default in code is `STAGE`. Whether active production credentials exist on the host server: Cannot confirm.

j. Why Paytm and why not

- Payments, Paytm. Final gateway. The app has wallet top-up and direct order payment. The wallet exists so checkout does not wait on a gateway for every order. Why Paytm: Paytm Split Settlement divides one payment between several merchant accounts. This capability fits the planned three-way split between shop, platform, and campus.
- Not PhonePe: used in the first design, then replaced by the owner with Paytm. No feature comparison was done.
- Any other gateway: No comparison was done.

> **Checked:** backend/main.py, supabase/migrations/011_paytm_three_way_split.sql, tables: payments, payment_splits, outlet_paytm_accounts, platform_settlement_account, college_settlement_accounts.

9. Other parts

Counter (POS) billing

Status: Built. Staff and shop admins create walk-in counter orders in `frontend/src/components/pos/PosCounterScreen.tsx`. Orders record cash, card, or counter UPI payments with zero gateway deductions. Database function `create_pos_order` allocates sequential order numbers and updates inventory.

Push notifications

Status: Partly built. Browser Web Push helper exists in `frontend/src/utils/webPush.ts`. Database table `push_subscriptions` stores browser endpoints and keys. Trigger `trg_observe_order_notifications` creates rows

in table `notifications`. Backend does not contain an active push service worker dispatching Web Push payloads.

MSG91

Status: Built. Backend contains fire-and-forget helper functions calling MSG91 WhatsApp outbound template APIs. Messages trigger on wallet top-up, order placed, order ready, and payment failure. Actual delivery requires valid credentials in variable `MSG91_AUTH_KEY`.

PWA install and offline use

Status: Built. Frontend registers a service worker caching application shells and menu data. Manifest file provides standalone application metadata and quick shortcuts. Network status listeners alert users when connectivity drops.

Menu photos and storage

Status: Built. Staff and shop admins upload food photos to Supabase Storage bucket `menu-images`. Image files are organized by outlet identifier. Backend route `/media/upload-menu-image` provides an image upload endpoint with local directory fallback.

> **Checked:** frontend/src/components/pos/PosCounterScreen.tsx, frontend/src/utils/webPush.ts, backend/main.py, frontend/public/sw.js, backend/routers/media.py, storage.buckets.

10. Hosting and settings

Status: Built.

What runs where on Render

From blueprint file `render.yaml`: 1. `vfoods-api`: Python web service running FastAPI backend in Singapore region. 2. `vfoods`: Static site running Vite React PWA frontend on global CDN. 3. `v-foods-landing`: Static site running Vite React landing page on global CDN.

Build and start commands

- `vfoods-api`: Build command: `pip install -r requirements.txt` Start command: `uvicorn main:app --host 0.0.0.0 --port $PORT`
- `vfoods`: Build command: `npm install && npm run build` Publish path: `dist`
- `v-foods-landing`: Build command: `npm install && npm run build` Publish path: `dist`

Environment variable names and where each is set

- Frontend variables (set in Render blueprint and `.env.local`): `VITE_API_URL` `VITE_SUPABASE_URL` `VITE_SUPABASE_ANON_KEY` `VITE_VAPID_PUBLIC_KEY`
- Backend variables (set in Render service dashboard and `.env`): `PYTHON_VERSION` `SUPABASE_URL` `SUPABASE_SERVICE_ROLE_KEY` `SUPABASE_JWT_SECRET` `PAYTM_MID` `PAYTM_MERCHANT_KEY` `PAYTM_WEBSITE` `PAYTM_CHANNEL_ID` `PAYTM_INDUSTRY_TYPE_ID` `PAYTM_ENV` `PHONEPE_MERCHANT_ID` `PHONEPE_SALT_KEY` `PHONEPE_SALT_INDEX` `PHONEPE_ENV` `PHONEPE_BASE_URL` `WEBHOOK_BASE_URL` `FRONTEND_URL` `QR_SECRET` `MSG91_AUTH_KEY` `MSG91_WHATSAPP_INTEGRATED_NUMBER` `MSG91_OTP_TEMPLATE_ID` `MSG91_SENDER` `SMTP_HOST` `SMTP_PORT` `SMTP_USER` `SMTP_PASSWORD` `SMTP_FROM`

> **Checked:** render.yaml, backend/render.yaml, backend/.env.example, frontend/.env.example.

11. Built and not built

Feature	Status	Where traced
Wallet top-up via Paytm	Built	`backend/main.py`, database function `verify_and_record_payment`
Wallet top-up via PhonePe	Built	`backend/main.py`, database function `credit_wallet`
Wallet order payment	Built	`frontend/src/App.jsx`, database function `place_order_wallet`
Direct gateway checkout via Paytm	Built	`backend/main.py`, database function `verify_and_record_payment`
Direct gateway checkout via PhonePe	Built	`backend/main.py`, database function `finalize_gateway_order`
Three-way split calculation	Built	Database function `calculate_order_split`, table `payment_splits`
Paytm webhook signature check	Built	`backend/main.py` function `_paytm_verify_checksum`
Paytm webhook duplicate idempotency	Built	Database function `verify_and_record_payment`
Paytm order status polling	Not built	`backend/main.py` (no query polling route exists)
Paytm automated gateway refunds	Not built	`backend/main.py` (code contains TODO comment)
Wallet cancellation refunds	Built	`backend/main.py`, database function `credit_wallet`
Counter POS billing screen	Built	`frontend/src/components/pos/PosCounterScreen.tsx`, function `create_pos_order`
Counter POS order voiding	Built	Database function `void_pos_order`
POS receipt generation	Built	`frontend/src/utils/receiptGenerator.js`
Daily sequential order tokens	Built	Database function `allocate_order_number`, table `outlet_daily_counters`
Realtime live order updates	Built	`frontend/src/App.jsx`, publication `supabase_realtime`
Realtime menu catalog sync	Built	`frontend/src/App.jsx`, table `menu_items`
Kitchen Display System (KDS)	Built	`frontend/src/pages/staff/ShopStaffDashboard.tsx`
Dish management and pricing	Built	`frontend/src/components/forms/MenuItemFormDialog.jsx`, table `menu_items`
Menu change audit trail	Built	Database trigger `trg_log_menu_item_changes`, table `menu_items_audit`
Food photo cloud storage	Built	`frontend/src/utils/menuStorage.ts`, Supabase Storage bucket `menu-images`
Scheduled pickup time slots	Built	`frontend/src/App.jsx`, table `pickup_slots`, function `generate_daily_pickup_slots`
Promotional discount coupons	Built	`frontend/src/App.jsx`, tables `coupons`, `coupon_redemptions`

Feature	Status	Where traced
Food item reviews and ratings	Built	`frontend/src/App.jsx`, table `item_ratings`, trigger `check_item_rating_order_status`
Customer loyalty tracking	Partly built	Table `loyalty_progress` exists, UI connection minimal
Four-tier role dashboards	Built	`frontend/src/App.jsx`, database function `get_my_role`, table `profiles`
Staff and shop admin invites	Built	Database functions `create_invite`, `accept_invite`, `backend/main.py`
Email sign-up with OTP	Built	`frontend/src/App.jsx`, function `register_manual_user`, `backend/routers/auth.py`
Google OAuth sign-in	Built	`frontend/src/App.jsx`, trigger `_handle_new_user`, function `complete_google_profile`
Phone number availability check	Built	Database function `check_mobile_availability`
WhatsApp alerts via MSG91	Built	`backend/main.py` functions `_send_whatsapp`, `_notify_order_placed`
Web Push notification delivery	Partly built	`frontend/src/utils/webPush.ts`, table `notifications`, backend sender missing
PWA offline caching	Built	`frontend/public/sw.js`, `frontend/public/manifest.json`
Native mobile binary app	Not built	`frontend/capacitor.config.json` exists, native platform folders missing
Stale order expiration cleaner	Built	Extension `pg_cron` job `expire-stale-awaiting-orders`, backend scheduler
Super Admin platform metrics	Built	Database function `get_super_admin_overview`, `SuperAdminDashboard.tsx`
Outlet franchise analytics	Built	Database function `get_outlet_analytics`, `ShopAdminDashboard.tsx`
Event stall operational mode	Built	Table `settings`, function `create_pending_gateway_order`, `place_order_wallet`

> **Checked:** frontend/src, backend/main.py, supabase/migrations, live database schema.

12. Open questions

1. Cannot confirm if live production merchant credentials exist in Paytm. Cannot confirm production Merchant ID, Merchant Key, or Channel ID values.
2. Cannot confirm whether the public webhook callback URL is registered in the Paytm merchant dashboard.
3. Cannot confirm whether active MSG91 API keys and approved WhatsApp message templates exist.
4. Cannot confirm if working SMTP credentials exist on Render for email OTP delivery.
5. Cannot confirm whether Google OAuth credentials in Supabase Auth are active in production.
6. Cannot confirm if any physical thermal printer hardware has been tested with POS receipt output.

> **Checked:** backend/main.py, render.yaml, Supabase project configuration.